Retrieval and Retry Mechanism

A code-review study guide for the Azure Cost Insight RAG pipeline

Prepared from the current repository source on 2026-09-30. This guide focuses on the runtime path that selects evidence, decides whether it is safe enough, retries retrieval when it is not, and constrains answer generation.

Review purpose

Use this document to explain the design in a code review. It distinguishes **retrieval retries** from **generation repair**: the first changes how evidence is found; the second fixes malformed or unsafe structured output without fetching new evidence.

Key safety property: the application does not use RAGAS at live request time. It uses deterministic checks to decide whether retrieval is adequate. RAGAS remains an offline evaluation tool.

1. One-page mental model

Step	Component	What it does	Can it call an LLM?
1	Inventory scope	Finds the exact named Azure resource, or safely abstains if it is absent.	No
2	Vector retrieval	Embeds the question and asks Chroma for child chunks; restores unique parent chunks.	Embedding model only
3	Rerank	Keeps exact resource evidence and selects complementary decision/utilization/policy roles.	No
4	Quality gate	Checks resource-specific evidence roles needed for remove/downsize questions.	No
5	Retrieval retries	Expands candidate pool, then rewrites the retrieval query if quality is still inadequate.	No
6	Structured generation	Asks Ollama or Gemini for strict JSON grounded in the chosen evidence plan.	Yes
7	Generation repair	Retries once only if the generated JSON fails validation.	Yes
8	Fallback	Returns safe evidence-only response if retrieval or generation cannot meet requirements.	No

The retry pipeline is deliberately bounded: initial retrieval, one broader candidate retrieval, then one focused-query retrieval. It does not loop forever and does not generate an answer until the deterministic quality gate passes.

2. Data shape: child chunks, parent chunks, and metadata

During ingestion, a source PDF is split into **parent chunks** (the readable evidence returned to the LLM) and smaller **child chunks** (the units stored and searched in Chroma). A child chunk carries a parent_id. This lets the retriever search small, specific passages but return the broader parent evidence with source/page/role metadata.

Code: ingestion/vector_store.py (lines 1-82)

Why this chunk matters: This is the Chroma persistence and query boundary. Notice the collection reset during re-ingestion, the parents.json companion store, raw-distance search, parent de-duplication, and the trace-only score conversion.

```
from langchain_chroma import Chroma
from ingestion.embedder import get_embedding_model
from config.settings import CHROMA_DB_PATH, COLLECTION_NAME
import json, os

PARENT_STORE_PATH = os.path.join(CHROMA_DB_PATH, "parents.json")


def store_chunks(parents, children):
    # Replace the collection so a re-ingestion updates parent metadata rather
    # than appending duplicate child chunks from a previous ingestion.
    try:
        Chroma(
            collection_name=COLLECTION_NAME,
            embedding_function=get_embedding_model(),
            persist_directory=CHROMA_DB_PATH,
        ).delete_collection()
    except Exception:
        # The first ingestion has no collection to remove.
        pass

    # langchain-chroma persists automatically when persist_directory is set.
    Chroma.from_documents(
        documents=children,
        embedding=get_embedding_model(),
        collection_name=COLLECTION_NAME,
        persist_directory=CHROMA_DB_PATH
    )

    os.makedirs(CHROMA_DB_PATH, exist_ok=True)
    with open(PARENT_STORE_PATH, "w", encoding="utf-8") as f:
        json.dump({
            "version": 2,
            "parents": {
                str(i): {"text": parent.page_content, "metadata": parent.metadata}
                for i, parent in enumerate(parents)
            },
        }, f)


def query_vector_store_with_metadata(query, top_k=3):
    """Return unique parent chunks together with their retrieval metadata."""
    db = Chroma(
        collection_name=COLLECTION_NAME,
        embedding_function=get_embedding_model(),
        persist_directory=CHROMA_DB_PATH
    )
    # Chroma's default distance metric is not guaranteed to satisfy
    # LangChain's [0, 1] relevance-score contract. Query raw distances and
    # convert them only for trace/display purposes; ranking remains Chroma's.
    hits = db.similarity_search_with_score(query, k=top_k)

    with open(PARENT_STORE_PATH, encoding="utf-8") as f:
        stored = json.load(f)
    # Support existing databases until the user re-runs ingestion.
    parents = stored.get("parents", stored) if isinstance(stored, dict) else {}

    seen, results = set(), []
    for hit, distance in hits:
        pid = str(hit.metadata.get("parent_id"))
        if pid not in seen and pid in parents:
            seen.add(pid)
            parent = parents[pid]
            if isinstance(parent, str):
                parent = {"text": parent, "metadata": {}}
            metadata = {**parent.get("metadata", {}), **hit.metadata}
            results.append({
                "id": pid,
                "text": parent["text"],
                "score": round(1 / (1 + max(float(distance), 0.0)), 6),
                "source": metadata.get("source"),
                "page": metadata.get("page"),
                "chunk_index": metadata.get("chunk_index"),
                "evidence_role": metadata.get("evidence_role"),
                "resource_names": metadata.get("resource_names", ""),
            })
    return results


def query_vector_store(query, top_k=3):
```

```
81      """Backward-compatible text-only retrieval helper."""
82      return [item["text"] for item in query_vector_store_with_metadata(query, top_k)]
```

Important review point: score = 1 / (1 + distance) is only a human-friendly trace/display value. Chroma's raw distance defines the initial similarity ordering; the application does not treat the converted score as a universal probability or a safety threshold.

3. Scope is resolved before retrieval

The application first obtains an Azure inventory and parses it into a consistent structure. If a question explicitly contains a hyphenated resource name, that name is authoritative. If it does not exist in the inventory, the system abstains rather than retrieve a similarly named resource and make a potentially unsafe recommendation.

Code: azure/resource_parser.py (lines 1-93)

Why this chunk matters: Exact-name scope resolution is the first protection against answering about the wrong resource. It uses regex boundaries so a partial name is not accepted as an exact match.

```
  1  import re
  2
  3
  4  def parse_resource(raw: dict) -> dict:
  5      return {
  6          "name": raw.get("name"),
  7          "type": raw.get("type"),          # e.g. Microsoft.Compute/virtualMachines
  8          "region": raw.get("location"),
  9          "sku": raw.get("sku", {}).get("name") if raw.get("sku") else None,
 10          "kind": raw.get("kind"),
 11          "tags": raw.get("tags", {}),
 12          "properties": raw.get("properties", {})
 13      }
 14
 15
 16  def parse_all_resources(raw_list: list) -> dict:
 17      """Parse and group resources by type to improve LLM clarity."""
 18      grouped = {}
 19      for r in raw_list:
 20          # Extract simple resource type name (e.g., 'virtualMachines' from 'Microsoft.Compute/
virtualMachines')
 21          full_type = r.get("type", "Unknown")
 22          resource_type = full_type.split("/")[-1] if "/" in full_type else full_type
 23
 24          if resource_type not in grouped:
 25              grouped[resource_type] = []
 26          grouped[resource_type].append(parse_resource(r))
 27
 28      return grouped
 29
 30
 31  def extract_resource_types_from_query(user_question: str) -> set:
 32      """Extract relevant resource types from user question to filter resources."""
 33      resource_patterns = (
 34          (r"\bvm\b|\bvirtual machines?\b", "virtualMachines"),
 35          (r"\bdisks?\b", "disks"),
 36          (r"\bstorage\b", "storageAccounts"),
 37          (r"\bdatabase\b|\bsql\b", "servers"),
 38          (r"\bnetwork\b", "virtualNetworks"),
 39          (r"\bnics?\b|\bnetwork interfaces?\b", "networkInterfaces"),
 40          (r"\bpublic ip(?: addresses?)?\b", "publicIPAddresses"),
 41          (r"\bload balancers?\b", "loadBalancers"),
 42          (r"\bapp service\b|\bapp service plans?\b", "serverfarms"),
 43          (r"\bweb apps?\b", "sites"),
 44          (r"\bcontainer registr(?:y|ies)\b|\bacr\b", "registries"),
 45          (r"\bkey vaults?\b", "vaults"),
 46      )
 47
 48      question_lower = user_question.lower()
 49      relevant_types = {
 50          resource_type
 51          for pattern, resource_type in resource_patterns
 52          if re.search(pattern, question_lower)
 53      }
 54      return relevant_types if relevant_types else None
 55
 56
 57  def resolve_resource_scope(resources: dict, user_question: str) -> dict:
 58      """Resolve an exact inventory scope before falling back to resource types.
 59
 60      An explicit resource name is authoritative. A resource-like name which is
 61      absent from the inventory is deliberately surfaced so the service can
 62      abstain instead of generating advice for unrelated resources.
 63      """
 64      question_lower = user_question.lower()
 65      exact_matches = {
 66          resource_type: [
 67              resource
 68              for resource in items
 69              if re.search(
 70                  rf"(?<![\w-]){re.escape(resource.get('name', '').lower())}(?![\w-])",
 71                  question_lower,
 72              )
 73          ]
 74          for resource_type, items in resources.items()
 75      }
 76      exact_matches = {
 77          resource_type: items for resource_type, items in exact_matches.items() if items
 78      }
 79      if exact_matches:
 80          return {"status": "matched", "resources": exact_matches, "candidates": []}
 81
```

```
    # Azure resource names commonly contain one or more hyphens. This is a
    # conservative signal for an explicit resource request, not a type query.
    candidates = re.findall(r"(?<![\w-])[a-z0-9]+(?:-[a-z0-9]+)+(?![\w-])", question_lower)
    if candidates:
        return {"status": "unknown_named", "resources": {}, "candidates": candidates}

    relevant_types = extract_resource_types_from_query(user_question)
    scoped_resources = resources if relevant_types is None else {
        resource_type: items
        for resource_type, items in resources.items()
        if resource_type in relevant_types
    }
```

Example: a question about tailspin-retry-ip-04 that is absent from the inventory returns an inventory-abstention response immediately. It does not query Chroma, call the answer LLM, or pretend an adjacent public IP is equivalent.

4. Exact resource matching and resource blocks

Semantic retrieval can return a parent chunk describing more than one resource. That is useful context, but it creates a risk: a fact about a disk might be incorrectly used as evidence to downsize the VM named next to it. The matching helpers prevent this leakage.

Code: rag/resource_matching.py (lines 1-67)

Why this chunk matters: The boundary-aware regex stops analytics-vm-01 from matching analytics-vm-01-data. resource_blocks() takes only the lines that belong to an allowed resource and ends the block when a different known resource begins.

```python
"""Token-aware helpers for matching inventory resource names in retrieved text."""

from __future__ import annotations

import re


def mentions_resource(text: str, resource_name: str) -> bool:
    """Match a complete Azure resource name, never a hyphenated prefix.

    For example, ``analytics-vm-01`` must not match
    ``analytics-vm-01-data``.
    """
    return bool(re.search(
        rf"(?<![\w-]){re.escape(resource_name.lower())}(?![\w-])",
        text.lower(),
    ))


def count_resource_mentions(text: str, resource_name: str) -> int:
    return len(re.findall(
        rf"(?<![\w-]){re.escape(resource_name.lower())}(?![\w-])",
        text.lower(),
    ))


def resource_blocks(
    text: str,
    allowed_names: list[str],
    inventory_names: list[str],
) -> list[str]:
    """Return only line groups belonging to the requested resource(s).

    Retrieved parent chunks can contain several inventory resources. A new
    block begins at an allowed resource, and ends before another known resource
    starts. This prevents an adjacent disk record from becoming VM evidence.
    """
    blocks: list[str] = []
    active_lines: list[str] = []
    active = False
    allowed_lower = [name.lower() for name in allowed_names]
    inventory_lower = [name.lower() for name in inventory_names]

    for raw_line in text.splitlines():
        line = re.sub(r"\s+", " ", raw_line).strip()
        if not line:
            continue
        is_allowed = any(mentions_resource(line, name) for name in allowed_lower)
        is_other_resource = any(
            mentions_resource(line, name)
            for name in inventory_lower
            if name not in allowed_lower
        )
        if is_allowed:
            if active_lines:
                blocks.append(" ".join(active_lines))
            active_lines, active = [line], True
        elif is_other_resource:
            if active_lines:
                blocks.append(" ".join(active_lines))
            active_lines, active = [], False
        elif active:
            active_lines.append(line)

    if active_lines:
        blocks.append(" ".join(active_lines))
    return blocks
```

How to explain it: a parent chunk is like a page containing several customer records. resource_blocks() cuts out only the record for the customer the user asked about before quality checks or answer planning inspect it.

5. Retrieval and deterministic reranking

retrieve_best_practice_contexts() is the main retrieval function. It asks Chroma for more candidates than the final top_k, then re-ranks them deterministically. This is not an LLM reranker: it is fast, explainable Python logic.

Code: rag/retriever.py (lines 1-119)

Why this chunk matters: The retriever has three responsibilities: use built-in guidance when the database has not been ingested, get a candidate pool from Chroma, and rank resource-specific evidence with complementary evidence roles.

```
   1  import os
   2
   3  from config.settings import CHROMA_DB_PATH
   4  from ingestion.vector_store import query_vector_store_with_metadata
   5  from rag.resource_matching import count_resource_mentions, mentions_resource
   6
   7
   8  DEFAULT_COST_BEST_PRACTICES = """\
   9  - Rightsize virtual machines from measured CPU, memory, and utilization data; schedule non-production
workloads outside business hours when owners approve.
  10  - Review unattached disks and unused public IP addresses with their owners before removal.
  11  - Apply storage lifecycle policies only when resilience, retention, and retrieval requirements permit.
  12  - Do not downsize production capacity without measured demand, owner approval, and a rollback plan.
  13  """
  14
  15
  16  def _rerank_for_resources(contexts, resource_names, top_k, query=""):
  17      """Select complementary operational, policy, and decision evidence."""
  18      if not resource_names:
  19          return contexts[:top_k]
  20
  21      names = [name.lower() for name in resource_names]
  22      action_priority = _evidence_role_priority(query)
  23
  24      # A scoped question must not borrow a decision from a different resource.
  25      # Keep only exact-name candidates whenever any are available; generic
  26      # guidance remains a fallback when the vector store has no scoped evidence.
  27      scoped_candidates = [
  28          context for context in contexts
  29          if any(mentions_resource(context["text"], name) for name in names)
  30      ]
  31      if scoped_candidates:
  32          contexts = scoped_candidates
  33
  34      def rank(context):
  35          text = context["text"].lower()
  36          exact_mentions = sum(count_resource_mentions(text, name) for name in names)
  37          semantic_score = context["score"] if context["score"] is not None else 0
  38          return (exact_mentions > 0, exact_mentions, semantic_score)
  39
  40      ranked = sorted(contexts, key=rank, reverse=True)
  41      # First take the best matching context for each evidence role. A complete
  42      # action recommendation needs more than a single semantically similar PDF.
  43      selected, seen_sources = [], set()
  44      for role in action_priority:
  45          candidate = next(
  46              (
  47                  item for item in ranked
  48                  if (item.get("evidence_role") or _infer_evidence_role(item["text"])) == role
  49                  and item.get("source") not in seen_sources
  50              ),
  51              None,
  52          )
  53          if candidate:
  54              selected.append(candidate)
  55              seen_sources.add(candidate.get("source"))
  56          if len(selected) == top_k:
  57              return selected
  58
  59      for context in ranked:
  60          if context not in selected and context.get("source") not in seen_sources:
  61              selected.append(context)
  62              seen_sources.add(context.get("source"))
  63          if len(selected) == top_k:
  64              break
  65      return selected
  66
  67
  68  def _infer_evidence_role(text: str) -> str:
  69      lowered = text.lower()
  70      if any(marker in lowered for marker in ("known exceptions", "required follow-up", "rollback window",
 "retained")):
  71          return "exception"
  72      if "decision register" in lowered:
  73          return "decision"
  74      if any(marker in lowered for marker in ("p95", "unattached", "no reads", "utilization", "metrics",
"snapshot")):
  75          return "utilization"
  76      if any(marker in lowered for marker in ("approved", "deferred", "pending")):
  77          return "decision"
```

```
    return "policy"


def _evidence_role_priority(query: str) -> tuple[str, ...]:
    """Choose a general evidence mix based on the requested action wording."""
    question = query.lower()
    if any(term in question for term in ("delete", "remove")):
        return ("decision", "utilization", "policy", "exception")
    if any(term in question for term in ("downsize", "downgrade")):
        return ("decision", "exception", "utilization", "policy")
    return ("decision", "utilization", "policy", "exception")


def retrieve_best_practice_contexts(query, top_k=3, resource_names=None, candidate_multiplier=6):
    """Return traceable retrieved contexts, or local fallback guidance before ingestion."""
    parent_store_path = os.path.join(CHROMA_DB_PATH, "parents.json")
    if not os.path.isfile(parent_store_path):
        return [{
            "id": "built-in-guidance",
            "text": DEFAULT_COST_BEST_PRACTICES,
            "score": None,
            "source": "built-in-guidance",
            "page": None,
            "chunk_index": None,
        }]
    # Fetch extra semantic candidates before applying the deterministic,
    # resource-name-aware reranker. This avoids a broad multi-resource chunk
    # crowding out evidence for the explicitly requested resource.
    candidates = query_vector_store_with_metadata(
        query, top_k=max(top_k * candidate_multiplier, top_k)
    )
    return _rerank_for_resources(candidates, resource_names or [], top_k, query=query)


def retrieve_best_practices(query, top_k=3):
    """Backward-compatible text-only retrieval helper."""
    contexts = retrieve_best_practice_contexts(query, top_k=top_k)
    return "\n\n".join(context["text"] for context in contexts)
```

How the reranker chooses

For an explicitly scoped resource, if any candidate names that exact resource, generic candidates are discarded. Candidates are then sorted by: (1) whether they mention the resource, (2) how many exact mentions they contain, and (3) the trace score. It attempts to select one source for each role in a question-specific priority order, avoiding repeated sources where possible.

For removal questions, the preferred order is **decision, utilization, policy, exception**. For downsize questions it is **decision, exception, utilization, policy**. This reflects the operational need to know both whether action is allowed and whether the data supports it.

6. Deterministic retrieval quality gate

A vector similarity score alone cannot prove that a destructive action is safe. The quality gate checks coverage. It answers: did we retrieve evidence that names the requested resource, and does that evidence include the types of information needed for this action?

Code: rag/retrieval_quality.py (lines 1-112)

Why this chunk matters: This module contains the live quality policy. It is deterministic and performs no model call. It records evidence roles, source count, best trace score, and explicit failure reasons.

```
  1  """Deterministic, low-latency checks used before asking the generation LLM.
  2
  3  This module deliberately does not call an evaluator model.  It checks whether
  4  the evidence selected for an action has the minimum operational coverage needed
  5  to make a safe answer.  RAGAS remains an offline evaluation tool, not a
  6  per-request production dependency.
  7  """
  8
  9  from __future__ import annotations
 10
 11  from typing import Any
 12
 13  from rag.resource_matching import resource_blocks
 14
 15
 16  def action_from_question(question: str) -> str:
 17      lowered = question.lower()
 18      if any(word in lowered for word in ("delete", "remove", "retire", "decommission", "get rid of")):
 19          return "remove"
 20      if any(word in lowered for word in ("downsize", "downgrade", "rightsize", "reduce capacity")):
 21          return "downsize"
 22      return "review"
 23
 24
 25  def evidence_role(context: dict[str, Any], text: str | None = None) -> str:
 26      # Roles for a named question must be derived from its resource-specific
 27      # block, rather than broad metadata from a parent chunk mentioning several
 28      # resources.
 29      declared = context.get("evidence_role") if text is None else None
 30      if declared:
 31          return str(declared)
 32      lowered = str(text if text is not None else context.get("text", "")).lower()
 33      if any(marker in lowered for marker in ("known exceptions", "required follow-up", "rollback window",
 "retained")):
 34          return "exception"
 35      if "decision register" in lowered or any(marker in lowered for marker in ("approved", "deferred",
"pending")):
 36          return "decision"
 37      if any(marker in lowered for marker in ("p95", "unattached", "no reads", "utilization", "metrics",
"snapshot")):
 38          return "utilization"
 39      return "policy"
 40
 41
 42  def assess_retrieval_quality(
 43      question: str,
 44      contexts: list[dict[str, Any]],
 45      resource_names: list[str],
 46      inventory_names: list[str] | None = None,
 47  ) -> dict[str, Any]:
 48      """Return explainable quality signals and a deterministic pass/fail verdict.
 49
 50      Exact resource evidence is essential for a scoped recommendation.  For
 51      destructive or capacity-reduction questions, the answer also needs a
 52      decision/exception plus an operational or policy safeguard.  This avoids
 53      treating a high vector similarity score as proof that an action is safe.
 54      """
 55      action = action_from_question(question)
 56      inventory_names = inventory_names or resource_names
 57      matching: list[tuple[dict[str, Any], str]] = []
 58      for context in contexts:
 59          for block in resource_blocks(context.get("text", ""), resource_names, inventory_names):
 60              matching.append((context, block))
 61      roles = sorted({evidence_role(context, block) for context, block in matching})
 62      sources = sorted({str(context.get("source")) for context, _ in matching if context.get("source")})
 63      scores = [context.get("score") for context, _ in matching if isinstance(context.get("score"), (int,
float))]
 64
 65      issues: list[str] = []
 66      if not contexts:
 67          issues.append("No contexts were retrieved.")
 68      if resource_names and not matching:
 69          issues.append("No retrieved context names the requested resource.")
 70      if action == "remove":
 71          if not ({"decision", "exception"} & set(roles)):
 72              issues.append("Removal evidence lacks a decision or exception record.")
 73          if not ({"utilization", "policy", "exception"} & set(roles)):
 74              issues.append("Removal evidence lacks a utilization, policy, or exception safeguard.")
 75      elif action == "downsize":
 76          if not ({"decision", "exception"} & set(roles)):
 77              issues.append("Rightsizing evidence lacks a decision or exception record.")
 78          if "utilization" not in roles:
```

```
  79              issues.append("Rightsizing evidence lacks measured utilization evidence.")
  80
  81      return {
  82          "passed": not issues,
  83          "action": action,
  84          "requested_resources": resource_names,
  85          "matching_context_count": len(matching),
  86          "evidence_roles": roles,
  87          "source_count": len(sources),
  88          "sources": sources,
  89          "best_relevance_score": max(scores) if scores else None,
  90          "issues": issues,
  91      }
  92
  93
  94  def build_resource_focused_query(question: str, resource_names: list[str]) -> str:
  95      """Rewrite retrieval only; preserve the user's original question for generation."""
  96      action = action_from_question(question)
  97      names = ", ".join(resource_names) or "requested resource"
  98      if action == "remove":
  99          need = "decision approval or hold, utilization or dependency evidence, retention and rollback
safeguards"
 100      elif action == "downsize":
 101          need = "decision or exception record and measured utilization, capacity, owner approval, and
rollback safeguards"
 102      else:
 103          need = "resource-specific decision, utilization, policy, and exception evidence"
 104      return f"Resource: {names}. Requested action: {action}. Retrieve {need}. Original question: {
question}"
```

Passing conditions

Question type	Minimum evidence needed
Remove / delete	At least one decision or exception record AND at least one utilization, policy, or exception safeguard.
Downsize / downgrade	At least one decision or exception record AND measured utilization evidence.
Review	Named evidence is still captured, but no additional action-specific role combination is required.

Failure is expected and useful. A failed check does not mean Chroma is broken. It means the retrieved evidence does not yet support a safe answer for the requested action, so the retry sequence is allowed to try a better evidence set.

7. The retrieval retry sequence

The orchestration lives in analyze_resources(). Each attempt is logged and kept in retrieval_quality["attempts"], making the flow visible through API traces and server logs.

Code: rag/service.py (lines 1-208)

Why this chunk matters: This is the complete orchestration path: inventory resolution, initial retrieval, two bounded retrieval retries, quality abstention, structured generation, generation repair, telemetry aggregation, and the response trace.

```
"""Reusable, traceable RAG service used by the UI, API, and future evaluations."""

import logging
from time import perf_counter
from typing import Any

from azure.resource_fetcher import fetch_all_resources

from azure.resource_parser import parse_all_resources, resolve_resource_scope
from config.settings import COLLECTION_NAME, LLM_PROVIDER, OLLAMA_MODEL, RETRIEVAL_TOP_K
from rag.llm_client import get_llm_response
from rag.response_policy import (
    build_repair_prompt,
    build_structured_answer_prompt,
    deduplicate_decision_fields,
    extract_evidence_plan,
    fallback_answer,
    parse_structured_answer,
    render_answer,
    validate_structured_answer,
)
from rag.retriever import retrieve_best_practice_contexts
from rag.retrieval_quality import assess_retrieval_quality, build_resource_focused_query


# Uvicorn configures this logger with its console handler at INFO level. Using
# it makes retry diagnostics visible in `uvicorn` output without requiring an
# application-wide logging configuration.
logger = logging.getLogger("uvicorn.error")


def _log_retrieval_attempt(strategy: str, quality: dict[str, Any]) -> None:
    """Emit operational retry diagnostics without logging prompts or evidence text."""
    logger.info(
        "retrieval_quality strategy=%s passed=%s resources=%s matches=%s roles=%s issues=%s",
        strategy,
        quality["passed"],
        quality["requested_resources"],
        quality["matching_context_count"],
        quality["evidence_roles"],
        quality["issues"],
    )


def analyze_resources(
    question: str,
    resource_group: str = "demo-cost-lab",
    subscription: str | None = None,
    top_k: int = RETRIEVAL_TOP_K,
) -> dict[str, Any]:
    """Run the RAG flow and return an evaluation-friendly execution trace."""
    started_at = perf_counter()
    generation_calls: list[dict[str, Any]] = []
    raw_resources = fetch_all_resources(resource_group, subscription)
    resources = parse_all_resources(raw_resources)
    scope = resolve_resource_scope(resources, question)
    scoped_resources = scope["resources"]
    allowed_names = [
        resource["name"]
        for items in scoped_resources.values()
        for resource in items
    ]

    if scope["status"] == "unknown_named":
        requested_name = scope["candidates"][0]
        answer = (
            f"Insufficient evidence. {requested_name} was not found in the supplied inventory, "
            "so its ownership, dependencies, retention requirements, and deletion safety cannot be
determined."
        )
        contexts = []
        retrieval_quality = {
            "passed": False,
            "action": "review",
            "requested_resources": [requested_name],
            "matching_context_count": 0,
            "evidence_roles": [],
            "source_count": 0,
            "sources": [],
            "best_relevance_score": None,
```

```
            "issues": ["The requested resource is not present in the supplied inventory."],
            "attempts": [{"strategy": "inventory_abstention", "passed": False}],
        }
        prompt = None
        scope_rewrite_applied = False
        response_mode = "inventory_abstention"
        decision = {
            "decision": "insufficient_evidence",
            "resource": requested_name,
            "reason": ["The resource is not present in the supplied inventory."],
            "required_conditions": ["Confirm ownership, dependencies, retention, and deletion safety."],
            "recommended_next_step": "Locate the resource in the authoritative inventory before acting.
",
            "citation_ids": [],
        }
    else:
        all_inventory_names = [
            resource["name"] for items in resources.values() for resource in items
        ]
        contexts = retrieve_best_practice_contexts(
            question,
            top_k=top_k,
            resource_names=allowed_names,
        )
        retrieval_quality = assess_retrieval_quality(
            question, contexts, allowed_names, inventory_names=all_inventory_names
        )
        _log_retrieval_attempt("initial", retrieval_quality)
        attempts = [{"strategy": "initial", **retrieval_quality}]
        if not retrieval_quality["passed"]:
            logger.info("retrieval_retry strategy=expanded_candidate_rerank resources=%s",
allowed_names)
            contexts = retrieve_best_practice_contexts(
                question,
                top_k=top_k,
                resource_names=allowed_names,
                candidate_multiplier=12,
            )
            retrieval_quality = assess_retrieval_quality(
                question, contexts, allowed_names, inventory_names=all_inventory_names
            )
            _log_retrieval_attempt("expanded_candidate_rerank", retrieval_quality)
            attempts.append({"strategy": "expanded_candidate_rerank", **retrieval_quality})
        if not retrieval_quality["passed"]:
            logger.info("retrieval_retry strategy=resource_focused_query_rewrite resources=%s",
allowed_names)
            rewritten_query = build_resource_focused_query(question, allowed_names)
            contexts = retrieve_best_practice_contexts(
                rewritten_query,
                top_k=top_k,
                resource_names=allowed_names,
                candidate_multiplier=12,
            )
            retrieval_quality = assess_retrieval_quality(
                question, contexts, allowed_names, inventory_names=all_inventory_names
            )
            _log_retrieval_attempt("resource_focused_query_rewrite", retrieval_quality)
            attempts.append({"strategy": "resource_focused_query_rewrite", **retrieval_quality})
        retrieval_quality["attempts"] = attempts
        plan = extract_evidence_plan(
            question,
            allowed_names,
            contexts,
            inventory_names=all_inventory_names,
        )
        scope_rewrite_applied = False
        if not retrieval_quality["passed"]:
            logger.warning(
                "retrieval_quality_abstention resources=%s issues=%s",
                allowed_names,
                retrieval_quality["issues"],
            )
            decision = fallback_answer({
                "decision": "insufficient_evidence",
                "resources": allowed_names,
                "evidence": [],
                "required_conditions": [],
            })
            prompt = None
            response_mode = "retrieval_quality_abstention"
        else:
            prompt = build_structured_answer_prompt(question, allowed_names, contexts, plan)
            generated_result = get_llm_response(prompt)
            generated = generated_result["text"]
            generation_calls.append({"purpose": "initial_generation", **generated_result["usage"]})
            decision = parse_structured_answer(generated)
            valid, _ = validate_structured_answer(decision, plan, all_inventory_names)
            if not valid:
                scope_rewrite_applied = True
```

```
                repaired_result = get_llm_response(build_repair_prompt(generated, question, plan))
                repaired = repaired_result["text"]
                generation_calls.append({"purpose": "repair_generation", **repaired_result["usage"]})
                decision = parse_structured_answer(repaired)
                valid, _ = validate_structured_answer(decision, plan, all_inventory_names)
            if valid:
                response_mode = "structured_repair" if scope_rewrite_applied else "structured"
            else:
                decision = fallback_answer(plan)
                response_mode = "evidence_fallback"
        answer = render_answer(deduplicate_decision_fields(decision))

    def aggregate_usage() -> dict[str, Any]:
        """Aggregate only RAG application calls; Judge calls never reach here."""
        base = generation_calls[0] if generation_calls else {}
        token_fields = ("input_tokens", "output_tokens", "total_tokens")
        totals = {}
        for field in token_fields:
            values = [call.get(field) for call in generation_calls]
            totals[field] = sum(values) if values and all(isinstance(value, int) for value in values)
else None
        return {
            "provider": base.get("provider", LLM_PROVIDER),
            "deployment": base.get("deployment", "local" if LLM_PROVIDER == "ollama" else "api"),
            "model": base.get("model", OLLAMA_MODEL if LLM_PROVIDER == "ollama" else None),
            "generation_calls": len(generation_calls),
            **totals,
        }

    return {
        "answer": answer,
        "question": question,
        "resource_group": resource_group,
        "resources": resources,
        "scope": {
            "status": scope["status"],
            "matched_resources": allowed_names,
            "scope_rewrite_applied": scope_rewrite_applied,
            "response_mode": response_mode,
        },
        "retrieved_contexts": contexts,
        "retrieval_quality": retrieval_quality,
        "prompt": prompt,
        "decision": decision,
```

Attempt order

Attempt	Strategy	What changes	What does not change
0	inventory_abstention	Used only for an explicit resource name missing from inventory.	No vector or LLM call.
1	initial	Candidate pool = top_k x 6 (default); resource-aware rerank returns top_k.	User question and requested resource.
2	expanded_candidate_rerank	Candidate pool = top_k x 12, then same resource-aware rerank.	Original question; no LLM call.
3	resource_focused_query_rewrite	Retrieval query explicitly asks for missing decision/utilization/safety evidence; pool remains x 12.	Original user question is preserved for generation.
4	retrieval_quality_abstention	Safe fallback is rendered when attempt 3 still fails.	No generation LLM call.

The focused rewrite looks like: *Resource: analytics-vm-01. Requested action: downsize. Retrieve decision or exception record and measured utilization, capacity, owner approval, and rollback safeguards. Original question: ...* The rewrite is used only to search for evidence. The user-facing generation prompt still contains the original question.

8. From validated evidence to an answer

Retrieval success is not the final guard. The response policy extracts an evidence plan, requests strict JSON from the selected provider, parses it, and validates it against resource scope and decision constraints. This makes the LLM responsible for phrasing an answer, not for inventing the decision basis.

Code: rag/response_policy.py (lines 1-233)

Why this chunk matters: This complete module builds the evidence plan, encodes the output contract, validates it, produces a safe fallback, removes duplicated lines, and renders the human-readable answer.

```
  1  """Evidence-first response planning and validation for cost recommendations."""
  2
  3  import json
  4  import re
  5  from typing import Any
  6
  7  from rag.resource_matching import mentions_resource, resource_blocks
  8
  9
 10  DECISIONS = {"yes", "no", "approved_with_conditions", "needs_review", "insufficient_evidence"}
 11  CONDITION_MARKERS = (
 12      "approval", "approve", "owner", "dependency", "retention", "retain",
 13      "rollback", "snapshot", "load test", "reassess", "after ", "pending",
 14  )
 15  NEGATIVE_MARKERS = (
 16      "do not downsize", "do not downgrade", "no immediate", "hold deletion",
 17      "retain", "deferred", "pending", "not clearly over-provisioned",
 18  )
 19  APPROVED_MARKERS = ("approved", "remove after", "delete after", "can be removed after")
 20
 21
 22  def _clean_text(text: str) -> str:
 23      return re.sub(r"\s+", " ", text).strip()
 24
 25
 26  def _sentences(text: str) -> list[str]:
 27      return [
 28          _clean_text(part).strip(" -•")
 29          for part in re.split(r"\n+|(?<=[.!?])\s+", text)
 30          if _clean_text(part).strip(" -•")
 31      ]
 32
 33
 34  def _action_from_question(question: str) -> str:
 35      lowered = question.lower()
 36      if any(action in lowered for action in ("delete", "remove", "retire", "decommission", "get rid
of")):
 37          return "remove"
 38      if any(action in lowered for action in ("downsize", "downgrade", "rightsize", "reduce capacity")):
 39          return "downsize"
 40      return "review"
 41
 42
 43  def extract_evidence_plan(
 44      question: str,
 45      resource_names: list[str],
 46      contexts: list[dict],
 47      inventory_names: list[str] | None = None,
 48  ) -> dict[str, Any]:
 49      """Turn retrieved evidence into a compact, model-independent decision plan."""
 50      action = _action_from_question(question)
 51      names = [name.lower() for name in resource_names]
 52      evidence: list[dict[str, str]] = []
 53      conditions: list[str] = []
 54      inventory_names = [name.lower() for name in inventory_names or resource_names]
 55
 56      for context in contexts:
 57          for block in resource_blocks(context["text"], resource_names, inventory_names):
 58              for sentence in _sentences(block):
 59                  lowered = sentence.lower()
 60                  references_resource = any(mentions_resource(sentence, name) for name in names)
 61                  mentions_condition = any(marker in lowered for marker in CONDITION_MARKERS)
 62                  mentions_observation = any(marker in lowered for marker in ("unattached", "reads",
"writes", "recovery artifact", "retained"))
 63                  # A parent paragraph can describe a VM and an associated disk.
 64                  # Disk hygiene is not evidence for a VM capacity-reduction
 65                  # decision unless the VM itself is named in that sentence.
 66                  if action == "downsize" and "disk" in lowered and not references_resource:
 67                      continue
 68                  if not (references_resource or mentions_condition or mentions_observation):
 69                      continue
 70                  item = {"text": sentence, "citation_id": str(context["id"])}
 71                  if item not in evidence:
 72                      evidence.append(item)
 73                  if mentions_condition and sentence not in conditions:
 74                      conditions.append(sentence)
 75
 76      combined_evidence = " ".join(item["text"] for item in evidence).lower()
 77      question_lower = question.lower()
 78      # A required condition means an action cannot be taken "immediately" or
 79      # "today", even if it may become approved after that condition is met.
 80      if conditions and ("immediately" in question_lower or " today" in question_lower):
```

```
        decision = "no"
    # Prefer safety over optimistic interpretation when the selected evidence is mixed.
    elif any(marker in combined_evidence for marker in NEGATIVE_MARKERS):
        decision = "no"
    elif any(marker in combined_evidence for marker in APPROVED_MARKERS):
        decision = "approved_with_conditions"
    else:
        decision = "needs_review"

    if not evidence:
        decision = "insufficient_evidence"

    return {
        "action": action,
        "decision": decision,
        "resources": resource_names,
        "evidence": evidence[:5],
        "required_conditions": conditions[:4],
    }


def build_structured_answer_prompt(
    question: str,
    resource_names: list[str],
    contexts: list[dict],
    plan: dict[str, Any],
) -> str:
    """Ask the model for a constrained transport format, not free-form prose."""
    schema = {
        "decision": "yes | no | approved_with_conditions | needs_review | insufficient_evidence",
        "resource": "one exact allowed resource name",
        "reason": ["one to three evidence-backed statements"],
        "required_conditions": ["mandatory safeguards before the action"],
        "recommended_next_step": "one concise action",
        "citation_ids": ["ids from supplied evidence"],
    }
    return f"""You are an Azure cost-optimization response generator.
Return JSON only, with no Markdown and no explanation outside the JSON.

Question: {question}
Allowed resource names: {json.dumps(resource_names)}
Structured evidence plan: {json.dumps(plan)}
Retrieved evidence: {json.dumps(contexts)}
Required JSON schema: {json.dumps(schema)}

Rules:
- Use only an allowed resource name.
- Follow the evidence-plan decision. A pending, deferred, retained, or approval-required action is not immediately allowed.
- Include all material approval, dependency, retention, rollback, or load-test conditions from the evidence plan.
- Do not mention another resource, generic regional pricing, or an unsupported recommendation.
- For a yes/no question, the decision must answer it directly."""


def build_repair_prompt(draft: str, question: str, plan: dict[str, Any]) -> str:
    return f"""Return corrected JSON only. Repair this cost-optimization response so it follows the required schema and evidence plan.

Question: {question}
Evidence plan: {json.dumps(plan)}
Draft response: {draft}

Use the same JSON schema fields: decision, resource, reason, required_conditions, recommended_next_step, citation_ids.
Do not mention resources outside {json.dumps(plan['resources'])}."""


def parse_structured_answer(value: str) -> dict[str, Any] | None:
    cleaned = value.strip()
    if cleaned.startswith("```"):
        cleaned = cleaned.split("\n", 1)[-1].rsplit("```", 1)[0].strip()
    start, end = cleaned.find("{"), cleaned.rfind("}")
    if start < 0 or end < start:
        return None
    try:
        payload = json.loads(cleaned[start : end + 1])
    except json.JSONDecodeError:
        return None
    return payload if isinstance(payload, dict) else None


def validate_structured_answer(
    payload: dict[str, Any] | None,
    plan: dict[str, Any],
    all_inventory_names: list[str],
) -> tuple[bool, str]:
    if not payload:
        return False, "response was not a JSON object"
```

```
    if payload.get("decision") not in DECISIONS:
        return False, "invalid decision"
    if payload.get("resource") not in plan["resources"]:
        return False, "response used an out-of-scope resource"
    if plan["decision"] in {"no", "approved_with_conditions"} and payload["decision"] != plan[
"decision"]:
        return False, "response contradicted the evidence-plan decision"
    if not isinstance(payload.get("reason"), list) or not payload["reason"]:
        return False, "response omitted evidence-backed reasons"
    if plan["required_conditions"] and not payload.get("required_conditions"):
        return False, "response omitted mandatory conditions"

    text = json.dumps(payload).lower()
    for name in all_inventory_names:
        if name not in plan["resources"] and re.search(rf"(?<![\w-]){re.escape(name.lower())}(?![\w-])",
text):
            return False, f"response leaked out-of-scope resource {name}"
    return True, ""


def fallback_answer(plan: dict[str, Any]) -> dict[str, Any]:
    """Produce a safe evidence-grounded response if structured generation fails."""
    resource = plan["resources"][0] if plan["resources"] else "the requested resource"
    evidence = [item["text"] for item in plan["evidence"]]
    conditions = plan["required_conditions"]
    next_step = (
        "Complete the required conditions and reassess the action."
        if conditions else "Gather the required ownership and utilization evidence before acting."
    )
    return {
        "decision": plan["decision"],
        "resource": resource,
        "reason": evidence or ["The retrieved evidence is insufficient for a safe recommendation."],
        "required_conditions": conditions,
        "recommended_next_step": next_step,
        "citation_ids": [item["citation_id"] for item in plan["evidence"]],
    }


def deduplicate_decision_fields(payload: dict[str, Any]) -> dict[str, Any]:
    """Do not render identical evidence as both a reason and a condition."""
    conditions = payload.get("required_conditions")
    reasons = payload.get("reason")
    if not isinstance(conditions, list) or not isinstance(reasons, list):
        return payload
    normalized_conditions = {_clean_text(str(item)).lower() for item in conditions}
    payload["reason"] = [
        item for item in reasons
        if _clean_text(str(item)).lower() not in normalized_conditions
    ]
    return payload


def render_answer(payload: dict[str, Any]) -> str:
    labels = {
        "yes": "Yes.",
        "no": "No.",
        "approved_with_conditions": "Approved with conditions.",
        "needs_review": "Needs review.",
        "insufficient_evidence": "Insufficient evidence.",
    }
    lines = [labels[payload["decision"]], f"Resource: {payload['resource']}"]
    if payload.get("reason"):
        lines.append("Evidence:")
        lines.extend(f"- {item}" for item in payload["reason"])
    if payload.get("required_conditions"):
        lines.append("Required conditions:")
        lines.extend(f"- {item}" for item in payload["required_conditions"])
    lines.append(f"Next step: {payload.get('recommended_next_step', '')}")
    return "\n".join(lines)
```

Generation repair is different from retrieval retry

If the initial LLM result is not valid JSON, uses an unknown resource, contradicts a required no/conditional decision, omits reasons, or omits mandatory conditions, the service sends one repair prompt. It supplies the same question and evidence plan. It does **not** retrieve again. If repair fails, the deterministic fallback renders the evidence plan directly.

> This distinction makes failures easier to diagnose: a retrieval retry means the system lacked evidence coverage; a generation repair means it had sufficient evidence but received invalid presentation/structure from the model.

9. LLM provider and application telemetry

The application can use a local Ollama model or the Gemini API. The retrieval quality check itself is independent of this choice. Only structured generation and, if necessary, generation repair invoke the LLM.

Code: rag/llm_client.py (lines 1-107)

Why this chunk matters: The provider adapter normalizes response text and returns non-secret token counters. Ollama reads prompt_eval_count/eval_count; Gemini uses normalized LangChain usage metadata where available.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage
from langchain_ollama import ChatOllama
from config.settings import (
    GEMINI_MODEL,
    GOOGLE_API_KEY,
    LLM_PROVIDER,
    OLLAMA_BASE_URL,
    OLLAMA_MODEL,
    OLLAMA_NUM_PREDICT,
    OLLAMA_REQUEST_TIMEOUT,
    OLLAMA_TEMPERATURE,
)


def _non_negative_int(value):
    """Return a provider token counter only when it is safe to expose."""
    if isinstance(value, bool):
        return None
    try:
        number = int(value)
    except (TypeError, ValueError):
        return None
    return number if number >= 0 else None

def _response_text(content):
    if isinstance(content, str):
        return content

    if isinstance(content, list):
        text_parts = []
        for block in content:
            if isinstance(block, str):
                text_parts.append(block)
            elif isinstance(block, dict) and block.get("type") == "text":
                text_parts.append(block.get("text", ""))
        return "".join(text_parts)

    return str(content)

def get_llm_response(prompt: str) -> dict:
    """Generate text and return provider usage without returning the prompt.

    Usage is intentionally limited to aggregate counters.  It is application
    telemetry, not an evaluator trace, and never contains secrets or prompt
    contents.
    """
    if LLM_PROVIDER == "ollama":
        llm = ChatOllama(
            model=OLLAMA_MODEL,
            base_url=OLLAMA_BASE_URL,
            temperature=OLLAMA_TEMPERATURE,
            # Azure-cost answers should be concise. This also prevents an
            # accidental long generation from monopolising the local model.
            num_predict=OLLAMA_NUM_PREDICT,
            # Without a client timeout, a stalled local generation can hold
            # the API worker (and an evaluation run) indefinitely.
            client_kwargs={"timeout": OLLAMA_REQUEST_TIMEOUT},
        )
    elif LLM_PROVIDER == "gemini":
        if not GOOGLE_API_KEY:
            raise RuntimeError("GOOGLE_API_KEY is required when LLM_PROVIDER=gemini.")
        llm = ChatGoogleGenerativeAI(google_api_key=GOOGLE_API_KEY, model=GEMINI_MODEL)
    else:
        raise RuntimeError(
            f"Unsupported LLM_PROVIDER '{LLM_PROVIDER}'. Use 'ollama' or 'gemini'."
        )

    response = llm.invoke([HumanMessage(content=prompt)])
    response_metadata = getattr(response, "response_metadata", {}) or {}
    usage_metadata = getattr(response, "usage_metadata", {}) or {}

    if LLM_PROVIDER == "ollama":
        input_tokens = _non_negative_int(response_metadata.get("prompt_eval_count"))
        output_tokens = _non_negative_int(response_metadata.get("eval_count"))
        provider = "ollama"
        model = OLLAMA_MODEL
        deployment = "local"
    else:
        # LangChain normalizes Gemini usage into usage_metadata.  Keep this
        # tolerant because provider integrations have changed field names.
        input_tokens = _non_negative_int(
```

```
            usage_metadata.get("input_tokens", usage_metadata.get("prompt_token_count"))
        )
        output_tokens = _non_negative_int(
            usage_metadata.get("output_tokens", usage_metadata.get("candidates_token_count"))
        )
        provider = "gemini"
        model = GEMINI_MODEL
        deployment = "api"

    total_tokens = (
        input_tokens + output_tokens
        if input_tokens is not None and output_tokens is not None
        else _non_negative_int(usage_metadata.get("total_tokens"))
    )
    return {
        "text": _response_text(response.content),
        "usage": {
            "provider": provider,
            "deployment": deployment,
            "model": model,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens,
            "total_tokens": total_tokens,
        },
    }
```

What the API trace exposes

Field	Meaning
retrieved_contexts	Parent evidence chunks with id, text, source, page, chunk index, role, and display score.
retrieval_quality	Final assessment plus the complete attempt history and failure reasons.
scope	Named resource scope, whether structured generation repair was used, and final response mode.
application_usage	Only application generation calls: provider, model, deployment, number of calls, input/output/total token counters when provider data is available.
meta	Collection name, top_k, configured provider/model, and end-to-end latency.

Token values are not fabricated. If a provider does not return a reliable count, the value remains None. API keys, prompts, and raw provider secrets are never added to this telemetry.

10. What to look for in logs

The service logs only diagnostic metadata, not prompts or evidence text. A successful first retry might look like this:

```
INFO retrieval_quality strategy=initial passed=False resources=['analytics-vm-01'] matches=3 roles=['policy',
'utilization'] issues=['Rightsizing evidence lacks a decision or exception record.']
INFO retrieval_retry strategy=expanded_candidate_rerank resources=['analytics-vm-01']
INFO retrieval_quality strategy=expanded_candidate_rerank passed=True resources=['analytics-vm-01'] matches=3
roles=['decision', 'utilization'] issues=[]
```

Read this as: initial retrieval knew about the VM and had utilization data, but lacked a decision record. The broader candidate set found a decision record. The quality gate passed, so generation was allowed. That is a retrieval success after one retry, not an LLM answer retry.

11. Code-review checklist

1. Does the inventory resolver safely distinguish an exact known resource from an unknown hyphenated name?

2. Are vector-store parent/child identifiers and metadata stable after a re-ingestion?

3. Does resource_blocks() prevent neighboring resources in a parent chunk from becoming evidence?

4. Are evidence-role markers appropriate for the organization's documents and vocabulary?

5. Do remove and downsize policies require sufficient operational safeguards for the risk level?

6. Are candidate_multiplier and top_k tuned using offline evaluation rather than intuition?

7. Are retries bounded, logged, and distinguishable from generation repair?

8. Does the structured-answer validator reject out-of-scope resources and missing mandatory conditions?

9. Is the fallback response safe and useful when vector evidence or model output is inadequate?

10. Are telemetry fields free of prompts, credentials, and sensitive raw provider payloads?

12. Practical walkthrough: a downsize question

Question: *Can we downsize analytics-vm-01?*

Step 1. The inventory resolver matches analytics-vm-01 exactly and makes it the only allowed resource.

Step 2. Chroma retrieves child hits and restores distinct parent evidence chunks. The reranker prefers chunks that mention analytics-vm-01 and aims to include decision plus utilization evidence.

Step 3. The quality gate sees utilization but no decision/exception record. It returns passed=False and an explicit issue.

Step 4. The expanded-candidate retry searches twice as many candidates, then reranks again. It finds a FINOPS decision record plus utilization snapshot.

Step 5. The quality gate now sees decision and utilization. It passes.

Step 6. The evidence planner decides approved_with_conditions/no/needs_review from the actual evidence wording and extracts mandatory conditions.

Step 7. The LLM receives the original question, allowed name, evidence plan, retrieved evidence, and JSON contract. It returns a structured decision.

Step 8. If that JSON is invalid or uses another resource, one generation repair occurs. Otherwise the answer is rendered with evidence, conditions, and next step.

Appendix: Configuration that affects the flow

Code: config/settings.py (lines 1-31)

Why this chunk matters: These environment-driven settings determine vector-store location, final context count, provider choice, and local generation safety limits. Secrets are read from the environment and are not embedded in source.

```
1  from dotenv import load_dotenv
2  import os
3
4  load_dotenv()
5
6  GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
7  GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-3.6-flash")
8  LLM_PROVIDER = os.getenv("LLM_PROVIDER", "ollama").lower()
9  OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:latest")
10 OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
11 CHROMA_DB_PATH = os.getenv("CHROMA_DB_PATH", "./chroma_db")
12 COLLECTION_NAME = os.getenv("COLLECTION_NAME", "azure_best_practices")
13 EMBEDDING_MODEL = os.getenv("EMBEDDING_MODEL", "all-MiniLM-L6-v2")
14 AZURE_SUBSCRIPTION_ID = os.getenv("AZURE_SUBSCRIPTION_ID")
15 RETRIEVAL_TOP_K = int(os.getenv("RETRIEVAL_TOP_K", "3"))
16 OLLAMA_TEMPERATURE = float(os.getenv("OLLAMA_TEMPERATURE", "0"))
17 OLLAMA_REQUEST_TIMEOUT = int(os.getenv("OLLAMA_REQUEST_TIMEOUT", "60"))
18 OLLAMA_NUM_PREDICT = int(os.getenv("OLLAMA_NUM_PREDICT", "450"))
19 ALLOW_TRACE_RESPONSE = os.getenv("ALLOW_TRACE_RESPONSE", "false").lower() == "true"
20 RAGAS_EVALUATOR_MODEL = os.getenv("RAGAS_EVALUATOR_MODEL", "llama3.2:latest")
21 RAGAS_API_BASE_URL = os.getenv("RAGAS_API_BASE_URL", "http://127.0.0.1:8000")
22 ANALYSIS_API_BASE_URL = os.getenv("ANALYSIS_API_BASE_URL", "http://127.0.0.1:8000")
23 RAGAS_EVALUATOR_PROVIDER = os.getenv("RAGAS_EVALUATOR_PROVIDER", "gemini").lower()
24 RAGAS_MAX_CASES = int(os.getenv("RAGAS_MAX_CASES", "0"))
25 RAGAS_REQUEST_TIMEOUT = int(os.getenv("RAGAS_REQUEST_TIMEOUT", "60"))
26 RAGAS_MAX_RETRIES = int(os.getenv("RAGAS_MAX_RETRIES", "2"))
27 RAGAS_MAX_WAIT = int(os.getenv("RAGAS_MAX_WAIT", "8"))
28 RAGAS_JUDGE_CONCURRENCY = int(os.getenv("RAGAS_JUDGE_CONCURRENCY", "4"))
29 RAGAS_CORRECTNESS_TIMEOUT = int(os.getenv("RAGAS_CORRECTNESS_TIMEOUT", "120"))
30 RAGAS_CORRECTNESS_MAX_RETRIES = int(os.getenv("RAGAS_CORRECTNESS_MAX_RETRIES", "4"))
31 RAGAS_CORRECTNESS_MAX_WAIT = int(os.getenv("RAGAS_CORRECTNESS_MAX_WAIT", "20"))
```

For a code review, the most consequential retrieval setting is RETRIEVAL_TOP_K. It controls the final count of parent contexts after reranking. candidate_multiplier is supplied by the service: 6 for initial retrieval, 12 for each retrieval retry.

Closing summary

This RAG design treats retrieval as an evidence-selection problem, not merely a nearest-neighbor lookup. It strengthens an initial semantic search with exact resource scope, role-aware deterministic reranking, deterministic quality criteria, bounded retries, structured generation, validation, and safe fallbacks. Each stage supplies observability so a reviewer can answer: *What did the system retrieve, why did it retry, and why was it allowed to answer?*